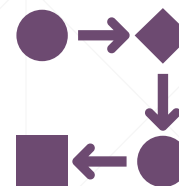




流数据转换

北京理工大学计算机学院
金旭亮

概述



使用Stream API处理数据，往往需要对原始数据进行特定的加工处理，之后，再转换为另一种类型的数据。

1

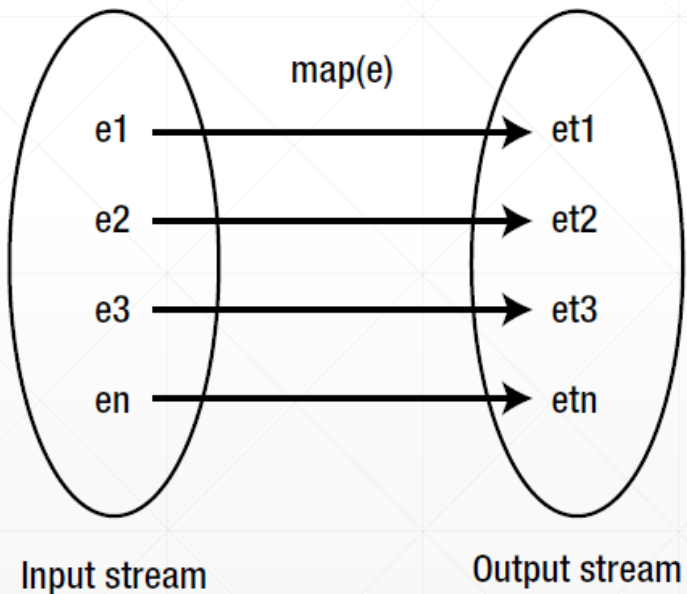
这种转换工作，如果位于数据处理管线的中间，则称为“Map”，Stream API提供了map和flatmap两个方法完成这个任务。

2

这种转换工作，如果位于数据处理管线的末端，Stream API提供了“reduce（归约）”和“collect（收集）”两个方法完成这个任务。

转换

使用`map()`方法可以把一种类型的数据经过特定的加工与处理之后，转换为另一种类型的数据。



`IntStream`类中定义的`map`函数：

```
IntStream map(IntUnaryOperator mapper);
```

`Stream<T>`接口中定义的`map`函数：

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper);
```

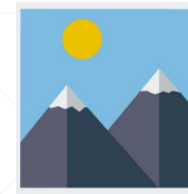
上述两个`map`方法的参数，都是函数式接口类型，因此都可以接收一个Lambda表达式。

map示例

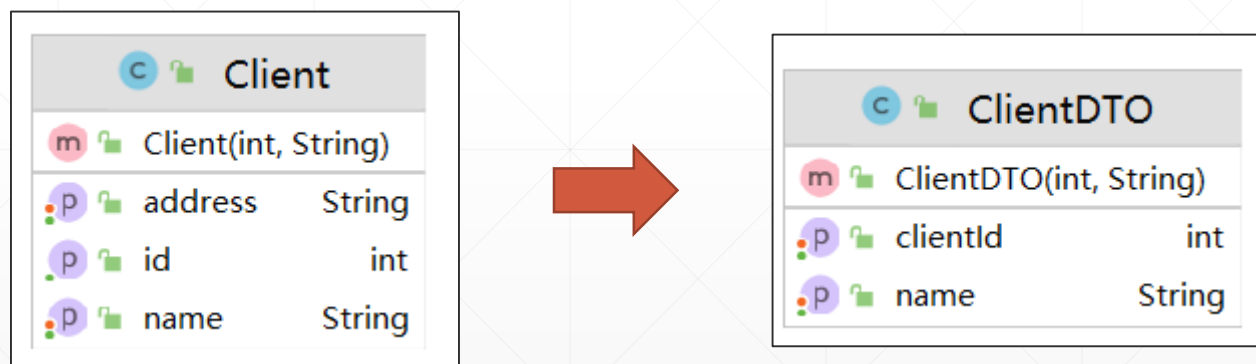
```
static void useMap() {  
    //计算1~5的整数平方值  
    IntStream.rangeClosed(1, 5)  
        .map(n -> n * n)  
        .forEach(System.out::println);  
  
    //将字符串集合中的所有字符串, 批量转换为小写  
    List<String> words = new ArrayList<>();  
    words.add("apple");  
    words.add("oranage");  
    //转换为小写  
    Stream<String> lowercaseWords = words.stream()  
        .map(String::toLowerCase);  
    lowercaseWords.forEach(System.out::println);  
}
```

TransformStreamDemo.java

Map的实际应用场景



在实际开发中，经常需要将数据库中的数据记录，抽取部分字段值，传给客户端，这种封装了发给客户端信息的对象，通常称为“**DTO（数据传输对象）**”，DTO对象通常出现在业务逻辑层与表示层中：



直接对应底层数据
存储结构

多用于业务逻辑层
或表示层代码

使用Stream API的map方法，可以很方便地完成这种DTO转换工作：

```
public class DtoDemo {  
    public static void main(String[] args) {  
        //创建示例集合  
        var clients : List<Client> = Client.createExampleClients( count: 10);  
        clients.forEach(System.out::println);  
  
        System.out.println("Client集合转换为ClientDTO集合");  
        //定义转换函数  
        Function<Client, ClientDTO> toDTO =  
            client -> new ClientDTO(client.getId(), client.getName());  
        //完成转换工作  
        List<ClientDTO> dtos = clients.stream()  
            .map(toDTO)  
            .collect(Collectors.toList());  
        dtos.forEach(System.out::println);  
    }  
}
```

DtoDemo.java

JDK中的其他Map方法



JDK 中为Stream提供了mapToInteger、mapToDouble等快捷方式，主要针对原始数据类型而准备，可以将对象流转换为IntStream，DoubleStream等类型的流，这些流类型处理相应的原始数据类型的数据，由于避免了不必要的装箱与拆箱操作，性能较好。

“转换”对象不是“复制”对象

MapObjectIsSame.java

```
public class MapObjectIsSame {  
    public static void main(String[] args) {  
        var clients : List<Client> = List.of(new Client( id: 1, name: "Client1"),  
            new Client( id: 2, name: "Client2"),  
            new Client( id: 3, name: "Client3"));  
  
        var newClients : List<Client> = clients.stream()  
            //提取clients中的所有对象, 将其名字改为大写  
            .map(client -> {  
                client.setName(client.getName().toUpperCase());  
                return client;  
            }).collect(Collectors.toList());  
  
        //两个List, 其实引用相同的对象  
        System.out.println(newClients.get(0) == clients.get(0)); //true  
        //以下两句输出, 完全一样  
        System.out.println(clients);  
        System.out.println(newClients);  
    }  
}
```

使用map转换对象类型，处理的就是原始的数据对象，之所以这样处理，是为了减少不必要的对象复制，减少对内存的占用。

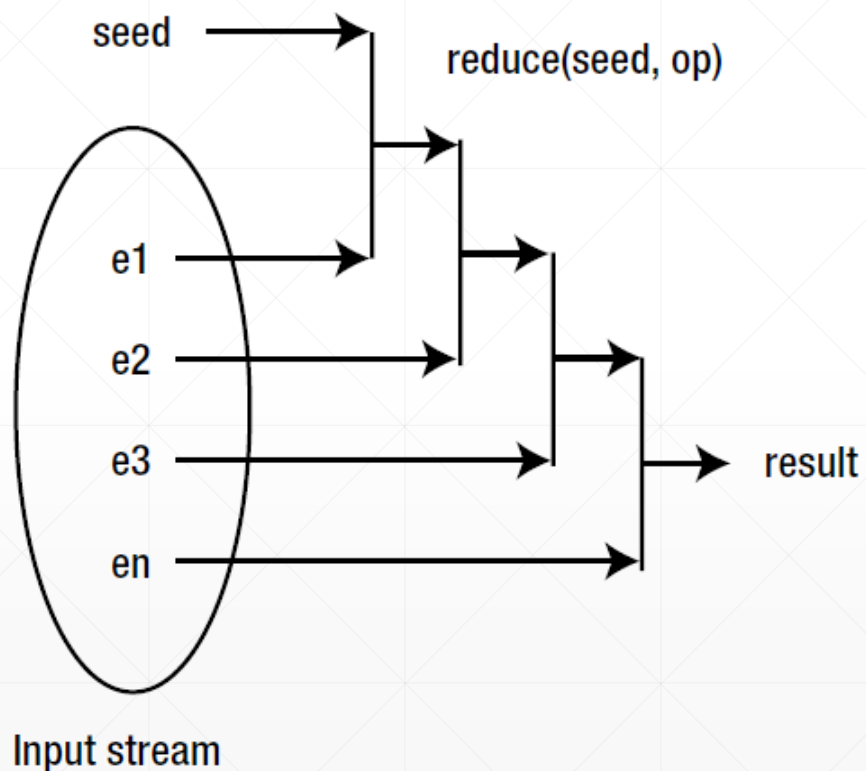
```
Run: MapObjectIsSame x  
"C:\Program Files\Java\jdk-17\bin\java.exe" "-  
true  
[Client{Id=1, Name='CLIENT1', Address='null'},  
[Client{Id=1, Name='CLIENT1', Address='null'},  
  
Process finished with exit code 0
```

flatMap

如果生成的结果是“元素是流的流（stream of stream）”，则需要使用 flatMap() 将形成嵌套关系的两个流“合成为一个”。

```
static void useFlatMap(){
    var words : List<String> = List.of("Hello", "World");
    words.stream() Stream<String>
        .map(word -> word.split( regex: "")) Stream<String[]>
        //将每个String[]转换为Stream<String>, 再展平
        //得到Stream<String>
        .flatMap(Arrays::stream) Stream<String>
        .distinct() //消除重复元素
        .forEach(System.out::println);
}
```

归约(Reduce)



所谓“**归约 (Reduce)**”，就是将流中所有元素反复组合起来，得到一个单一的值。

Stream API提供了一个reduce方法，用于实现归约操作。

理解归约-1

比如我们要计算一个集合中所有元素的和，使用循环方式，代码是这样的.....

//使用传统的循环方式来求和

```
static void sumByLoop() {  
    var nums : List<Integer> = List.of(4, 5, 3, 9);  
    int sum = 0;  
    for (var num : nums) {  
        sum += num;  
    }  
    System.out.println(sum);  
}
```

示例：StreamReduceDemo.java

理解归约-2

使用归约的方法实现

//使用Reduce方式求和

```
static void sumByReduce() {  
    var nums : List<Integer> = List.of(4, 5, 3, 9);  
  
    var sum : Integer = nums.stream()  
        .reduce( identity: 0, (a, b) -> a + b);  
    System.out.println(sum);//21  
}
```

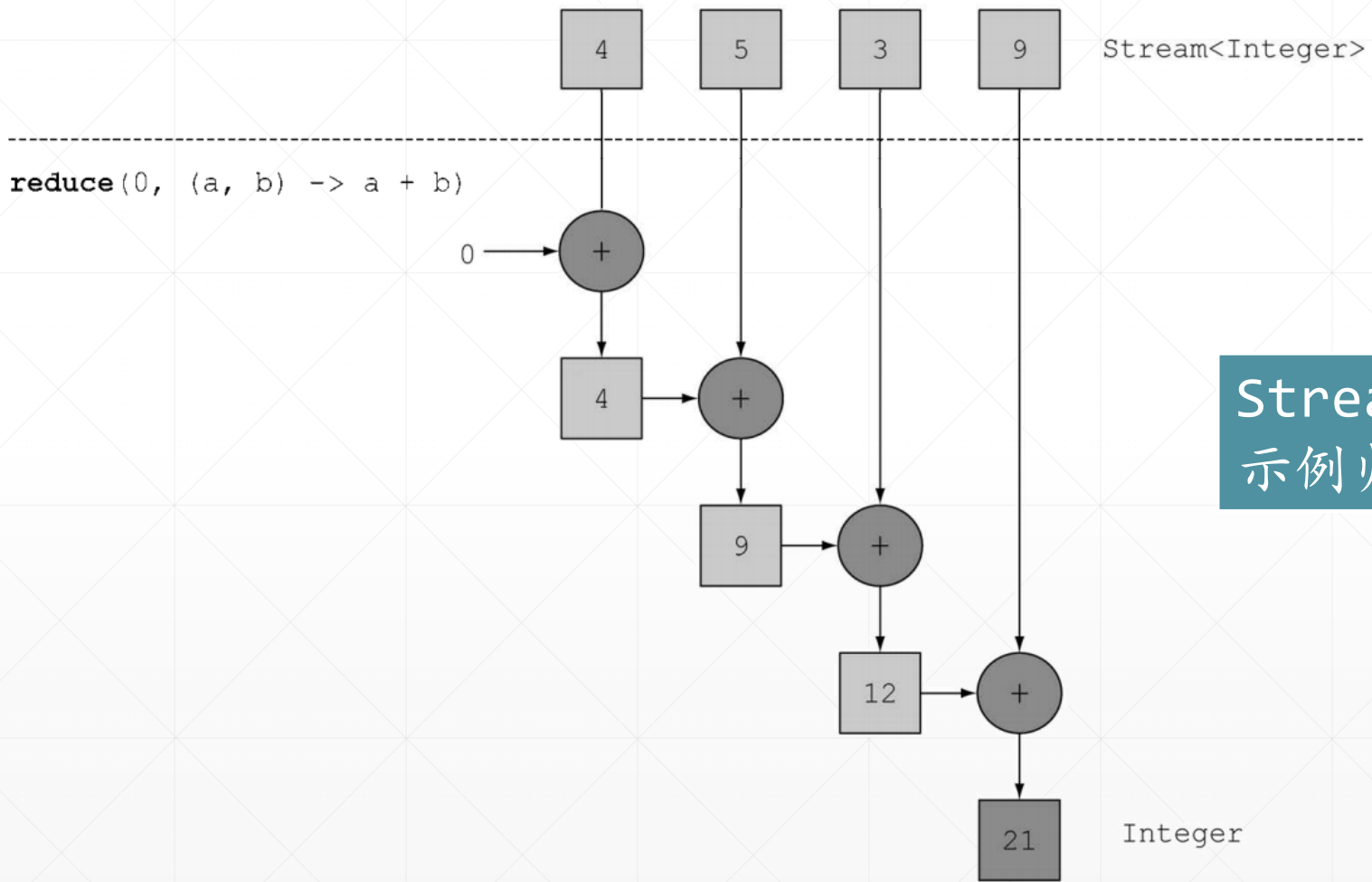
//用方法引用简写

```
sum = nums.stream().reduce( identity: 0, Integer::sum);  
System.out.println(sum);
```

示例: StreamReduceDemo.java

上述reduce方法的第一个参数是初始值，称为seed，第二个参数是一个实现了BinaryOperator<T>的Lambda表达式，它接收的第一个数据元素就是seed，之后，顺次接收流中的每个元素，每次都生成一个中间结果，将它与流中的下一个元素相加，直到遇到流中的最后一个元素，返回最终结果。

数值流

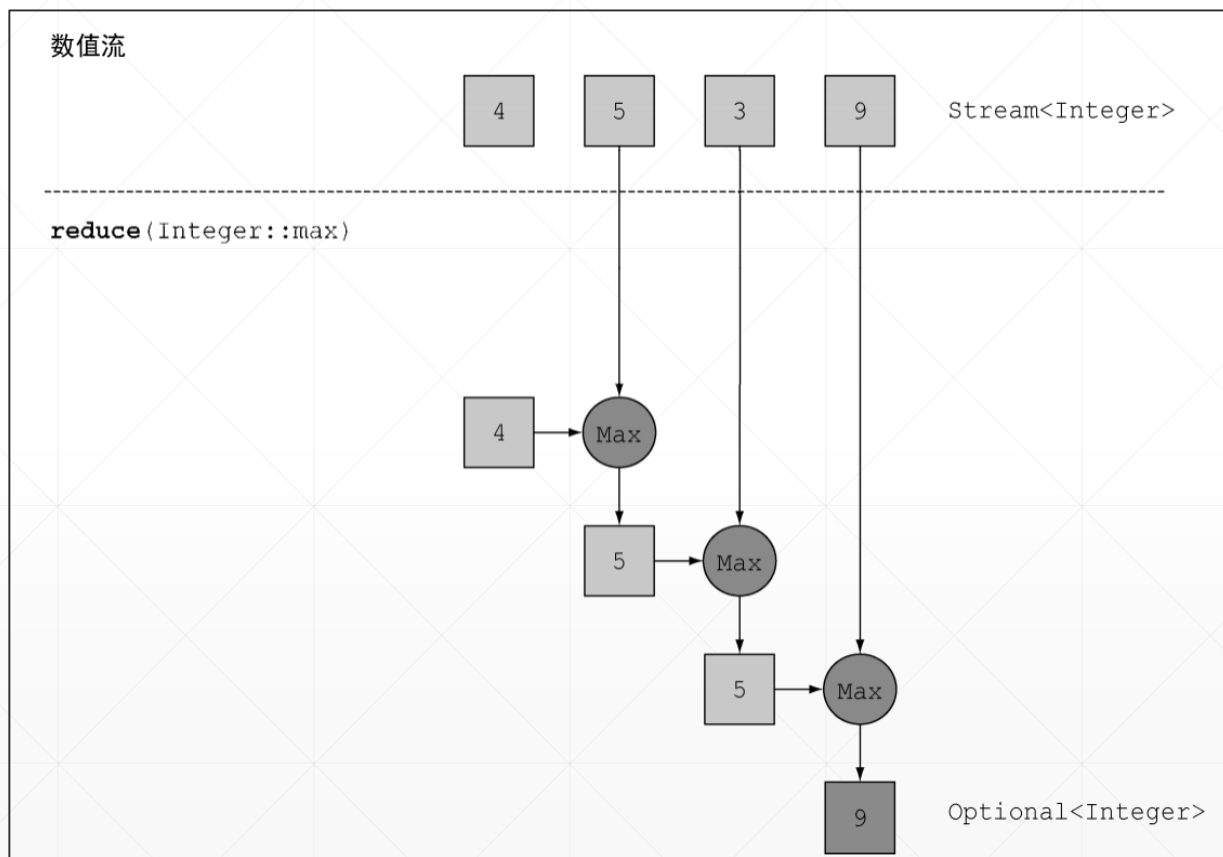


StreamReduceDemo.java
示例归约过程的图解

练习



你能否使用reduce方法，找出一个集合中的最大元素？



参考答案: GetMax.java

下一讲



Optional 类型